TELESCOPE

Project Proposal - Analysis & Design Phase Blueprint

Egyptian telecom market intelligence: sentiment analysis and hybrid recommendations

DOMAIN	CAPABILITIES	PROVIDERS	PLANNING WINDOW	VERSION
Egyptian Telecom Market Intelligence	Sentiment Analysis & Hybrid Recommendations	WE, Orange, Vodafone, Etisalat	26 Sep - 4 Dec 2026 (10 weeks)	v1.0, 27 Oct 2026

1 Analysis

1.1 Problem Statement & Needs

Problem. Egyptian telecom customers compare providers and offers using scattered information: app reviews, personal experiences, and offer descriptions. Without a unified analysis workflow, these sources are hard to compare consistently.

Solution. TeleScope is a data-driven platform that unifies Google Play reviews, survey responses, and offer data for WE, Orange, Vodafone, and Etisalat. It analyzes customer sentiment, compares providers, summarizes offer characteristics, and recommends providers or offers that match the user's stated needs.

Primary needs

- Collect, clean, validate, and integrate data from the three confirmed sources.
- Analyze customer sentiment from app reviews.
- Compare providers using review, survey, and offer evidence.
- Match users with suitable offers through a hybrid rule-based + machine-learning approach.
- Present findings in a clear Streamlit interface.

Data caveat. Google Play reviews represent only app users who choose to post; survey findings depend on sample size and respondent coverage; offers change over time and are reported with collection timestamps where available.

1.2 Stakeholder Analysis

Stakeholder	What they need
Supervisor / evaluators	A clearly scoped project, defensible methodology, documented decisions, evaluation results, and a working demo.
Project team (6 members)	Repeatable collection and processing, shared data access, clear interfaces between components, a realistic plan.
Customers / platform users	Understandable provider comparisons and recommendations that reflect budget, quota, validity, and other preferences.
Data / business analysts	Comparable summaries, traceable sources, and visualizations of feedback, survey results, and offer differences.
Telecom providers (indirect)	May benefit from aggregated feedback insights. TeleScope is not an official provider system and does not use provider-internal data.

1.3 Feasibility Study

Dimension	Assessment
Technical	Established tools: Python, Airflow, PostgreSQL, Streamlit, and Microsoft Azure Blob Storage. Processing engine (Pandas vs. PySpark) is chosen by data volume.

Dimension	Assessment
Operational	Six-person team supported by shared storage, folder conventions, Git, and repeatable pipelines. Test storage access and network reliability early.
Economic	Open-source tools and existing team infrastructure first. Azure Blob Storage subscription, compute, and deployment costs are unconfirmed and must be checked before using any paid service.
Schedule	Ten weeks, 26 Sep - 4 Dec 2026. Work has started, so compare actual progress against the plan and prioritize an end-to-end product over optional features.

Key assumptions to verify: data can be collected lawfully and within platform rules; enough usable reviews, survey responses, and offer records are obtainable; a suitable deployment environment is available.

2 Design

2.1 Objectives & Scope

Overall objective. Design and implement TeleScope to analyze the Egyptian telecom market and help users compare providers and identify suitable offers using reviews, survey data, offer attributes, sentiment analysis, and a hybrid recommendation system.

Specific objectives

1. Build repeatable collection workflows for Google Play reviews, surveys, and offers for the four providers.
2. Store raw source data separately from cleaned and curated datasets.
3. Create a PostgreSQL data model supporting consistent provider, review, survey, and offer comparisons.
4. Develop and evaluate a sentiment-analysis component for review text.
5. Build a hybrid recommender: rule-based filters for hard constraints, then ML ranking of eligible offers.
6. Deliver a Streamlit interface for market insights, provider comparison, and offer recommendations.
7. Test the data workflow, analytical outputs, recommendation behavior, and final application.

In scope	Out of scope
• Google Play reviews for the four providers • Project survey responses • Telecom offer records and attributes • Collection, validation, cleaning, deduplication, integration • Azure Blob Storage, orchestration, PostgreSQL warehouse • Sentiment analysis and hybrid recommendations • Streamlit dashboards and comparison views • Testing, documentation, deployment prep, presentation	• Real-time streaming as a core requirement • Native Android or iOS application • Replacing providers' internal or customer-service systems • Guaranteed collection of every review or offer • Training a large language model from scratch • Measuring live network coverage or performance • Separate Flask backend (only if a distinct API need emerges)

Success criteria	Constraints
• All three source categories represented in the workflow, subject to availability • Documented schemas and basic quality checks • Warehouse supports comparable views across the four providers • Sentiment performance evaluated on a documented validation/test set • Recommendations obey essential user constraints, with a documented baseline and evaluation • Streamlit app demonstrates the main end-to-end user journey	• Ten-week delivery window • Uneven availability and quality of source data • Possible collection limits, interruptions, or language imbalance in reviews • Azure Blob Storage availability and network access • Limited time for model tuning and integration • Azure subscription access and cost not yet confirmed

2.2 Technical & System Architecture

The platform separates collection, raw storage, orchestration, processing, warehouse, analytics/ML, and the user interface. This supports traceability and lets the team swap storage or processing components without redesigning the whole application. Principle: build a reliable end-to-end path first, then expand volume and model complexity once the pipeline and data quality are stable.

1 Sources Google Play reviews, surveys, telecom offers	**2 Collection** Python collectors with source validation	**3 Raw storage** Microsoft Azure Blob Storage	**4 Orchestration** Apache Airflow schedules and monitors
5 Processing Python / Pandas: clean, dedupe, enrich	**6 Warehouse** PostgreSQL analytical data model	**7 Analytics & ML** Sentiment analysis + hybrid recommender	**8 Streamlit UI** Services layer, comparison, insights

Layer	Choice and notes
Collection	Python; separate source-specific collectors.
Object storage	Microsoft Azure Blob Storage.
Orchestration	Apache Airflow for scheduled, observable workflows.
Processing	Python and Pandas; PySpark only if volume or performance justifies it.
Warehouse	PostgreSQL with documented entities and relationships.
ML	Sentiment analysis plus the hybrid recommendation method.
Application	Streamlit with a Services layer and repository functions. Flask only if an independent REST API becomes a confirmed requirement.
Quality	Git, environment-variable configuration, validation checks, documented tests.

Storage zones

Raw	Processed	Curated
Source data as collected, preserving source context.	Cleaned, normalized, deduplicated records with consistent types and names.	Analysis-ready tables and features for the warehouse, dashboards, and ML.

Warehouse, recommendation, and security design

- Warehouse. Confirm the final schema against actual source fields. Preserve provider identity, source provenance, collection timestamps, review and survey attributes, and offer fields (provider_name, offer_title, offer_type, price_egp, quota_amount/unit, speed_limit/unit, validity_days, offer_description, target_customer, scraped_at, source_url). Do not assume every source supplies every field.
- Recommendation. Hybrid recommender: Step 1 applies rule-based filters for hard constraints (budget, quota, validity); Step 2 applies ML ranking to the eligible offers using validated features. The algorithm and metrics are chosen after data profiling; if preference data is insufficient, the rule-based baseline stays available and its limits are disclosed.
- Security and quality. Keep credentials out of source code, restrict access to Azure Blob Storage, validate incoming schemas, track source and collection time, handle duplicates, and log pipeline failures. Aggregate or anonymize survey data in public dashboards.

2.3 Timeline & Risk Matrix

Week	Focus
Week 1	Analyze problem, stakeholders, needs, and scope.
Week 2	Finalize architecture, data-model direction, collection strategy, and risks.
Week 3	Begin collection; configure Azure Blob Storage.
Week 4	Continue collection; check coverage; organize raw datasets.
Week 5	Clean and validate data; build and populate the initial warehouse.
Week 6	Implement and evaluate the initial sentiment-analysis model.
Week 7	Implement the hybrid recommendation component.
Week 8	Build Streamlit; connect data and model services.
Week 9	Test pipelines, models, recommendations, and integration; prepare deployment.
Week 10	Fix critical issues, finalize documentation, deploy, prepare the presentation.

Timeline clarification. The planning window runs ten weeks, from 26 Sep 2026 to 4 Dec 2026, which is the submission deadline. If progress falls behind, cut lower-priority work: weeks 8-10 are the most compressible, so protect the end-to-end demo first.

Risk matrix

Risk	Likelihood	Impact	Mitigation
Incomplete collection	Med/High	High	Monitor counts per provider, language, and source; retry safely; document gaps.
Data imbalance across providers	High	Med/High	Report coverage; avoid unsupported conclusions from unequal samples.
Poor data quality	High	High	Schema checks, deduplication, normalization, validation reports.
Azure Blob Storage / network outage	Medium	High	Test access early; document recovery and retry procedures.
Warehouse design changes	Medium	High	Profile sources early; version the schema and migrations.
Weak sentiment performance	Medium	High	Set a baseline; inspect representative and language-specific errors.
Insufficient recommendation signals	Med/High	Med/High	Keep rule-based matching; use only supported features; disclose limits.
Integration delays	Medium	High	Agree on interfaces early; test small end-to-end slices regularly.
Schedule compression	High	High	Prioritize required features; reserve the final week for critical fixes.

Open decisions to validate

- Azure Blob Storage subscription, budget, and access configuration.
- Final warehouse schema after profiling all source files.
- Sentiment model choice and evaluation protocol.
- Recommendation features and ML ranking method supported by the data.
- Deployment environment and any additional API requirements.